Exception Handling

Instructor:

Table of contents

Learning Approach

Introduction exception

- During the execution of a program, the computer will face the some of situations:
  - syntax error
  - logic algorithm error
  - runtime error
- An exception is an abnormal condition that arises[xuất hiện] in a code sequence at run time.
  - an exception is a run-time error
- Java's exception handling avoids the problems in the run-time.

Introduction exception (cont.)

- ✓ A program that requests a number from the user:
- ✓ Ex 1:

```
public class Exercise {
    public static void main(String[] args) {
        double side;
        Scanner scnr = new Scanner(System.in);
        System.out.println("Square Processing");
        System.out.print("Enter Side: ");
        side = scnr.nextDouble();
        System.out.println("\nSquare Characteristics");
        System.out.printf("Side: %.2f\n", side);
        System.out.printf("Perimeter: %.2f\n", side * 4);
    }
}
```

```
Square Processing
Enter Side: dS
Exception in thread "main" java.util.InputMismatchException
        at java.util.Scanner.throwFor(Unknown Source)
        at java.util.Scanner.next(Unknown Source)
        at java.util.Scanner.nextDouble(Unknown Source)
        at clc.btjb.unit04.les04.Exercise.main(Exercise.java:11)
```

Introduction exception (cont.)

- Runtime errors can be divided into low-level errors that involve violating constraints, such as:
  - dereference of a null pointer
  - out-of-bounds array access
  - divide by zero
  - attempt to open a non-existent file for reading
  - bad cast (e.g., casting an Object that is actually a Boolean to Integer)

Introduction exception (cont.)

- and higher-level, logical errors, such as violations of a function's precondition:
  - call to Stack's "pop" method for an empty stack
  - call to "factorial" function with a negative number
  - call to List's nextElement method when hasMoreElements is false

Introduction exception (cont.)

- **Ex 2:**

```
public class Lowlevel_Exception {
    public static void main(String[] args) {
        int x = 0, z = 5;
        int j = 3;
        int y = z / x;  // Exception: Divide By Zero

        int[] ar = new int[j];// Exception: Index Out Of Range
        ar[j] = 5;
        // suppose that a.txt does not exist
     BufferedReader input = new BufferedReader(
                                    new FileReader("a.txt"));
     // Exception: File Not Found
    }
}
```

Exception handling

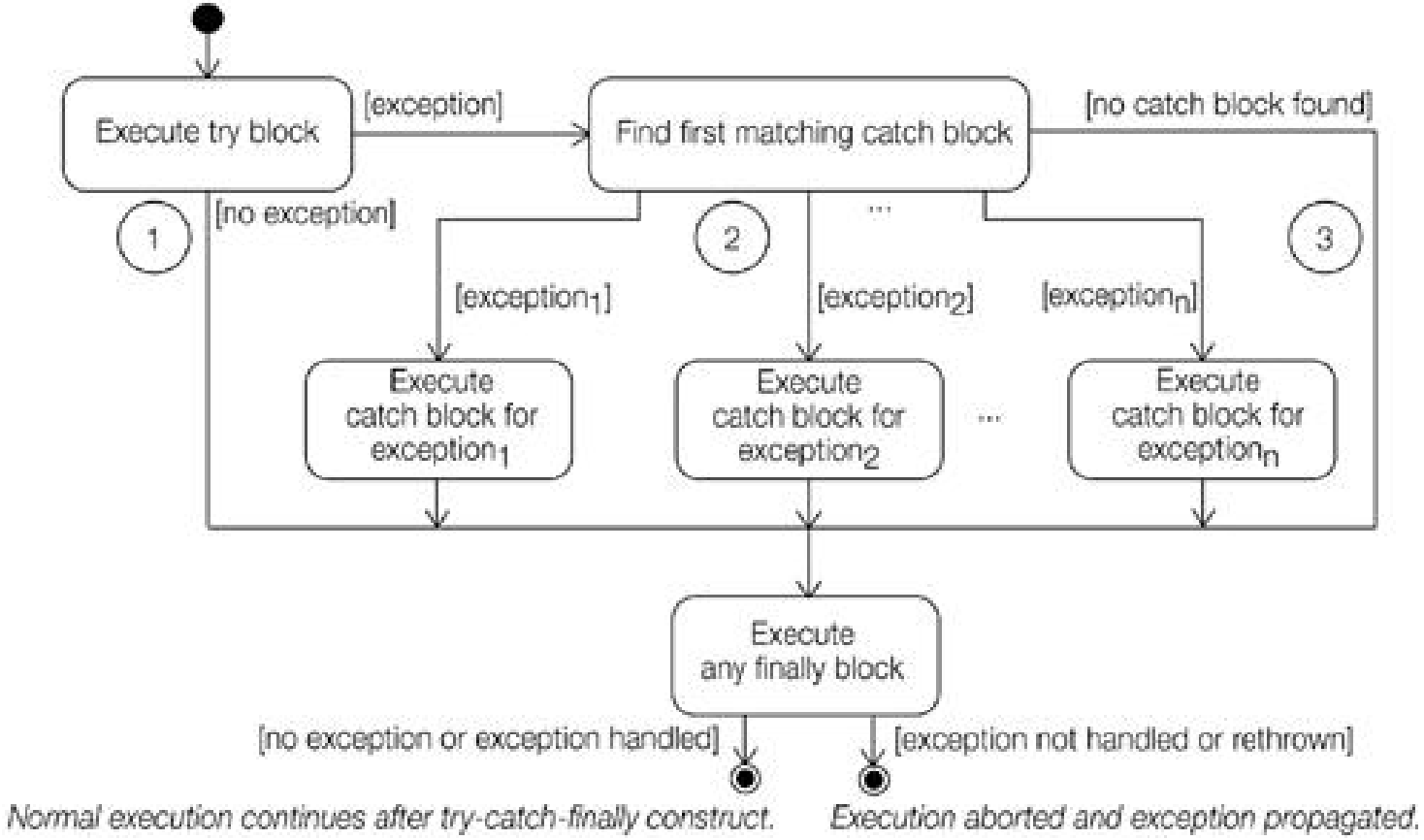
- When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error.
- Java exception handling is managed via five keywords: **try**, **catch**, **throw**, **throws**, and **finally**.
  - Program statements that you want to monitor for exceptions are contained within a **try** block.
  - Your code can catch this exception (using **catch**) and handle it in some rational manner.
  - To manually throw an exception, use the keyword **throw**. Any exception that is thrown out of a method must be specified as such by a **throws** clause. Any code that absolutely must be executed before a method returns is put in a **finally** block.

Exception handling (cont.)

- **This is the general form of an exception-handling block:**

```
try {
    // block of code to monitor for errors
}
catch (ExceptionType1 exOb) {
    // exception handler for ExceptionType1
}
catch (ExceptionType2 exOb) {
    // exception handler for ExceptionType2
}
// ...
[finally {
    // block of code to be executed before try block ends
}]
```

Exception handling (cont.)



Exception handling (cont.)

- **Solution ex1:**

```
try{
    side = scnr.nextDouble();
    System.out.println("\nSquare Characteristics");
    System.out.printf("Side: %.2f\n", side);
    System.out.printf("Perimeter: %.2f\n", side * 4);
}
catch(InputMismatchException ex)
{
  System.out.println(ex.getMessage());
  // method get message
}
```

Exception handling (cont.)

- ✓ **Solution ex2:**

```
public static void main(String[] args) {
    int x = 0, z = 5;
    int j = 3;
    int[] ar = new int[j];
    try {
        int y = z / x;
        ar[j] = 5;

        BufferedReader input = new BufferedReader(new
                                FileReader("a.txt"));
    } catch (Exception ex) {
       ex.getStackTrace();
      // Exception Method Call Stack Trace
    }
}
```

Exception handling (cont.)

- **Solution ex2:**

```
} catch (ArithmeticException ex1) {
  System.out.println(ex1.getStackTrace());
}
catch(ArrayIndexOutOfBoundsException ex2)
{
  System.out.println(ex2.getStackTrace());
}
catch(FileNotFoundException ex3)
{
  System.out.println(ex3.getStackTrace());
}
```

Exception handling (cont.)

- **Finally block**

```
Connection conn= null;
try {
   conn= get the db conn;
   //do some DML/DDL
} catch(SQLException ex)
{
} finally
{
   conn.close();
}
```

Using Throws and Throw Statements

- **Ex 3:** Imagine you have been assigned a task of finding a specific book, and then reading and explaining its contents to a class of students. The required sequence may look like:
  - ✓ Get the specified book
  - ✓ Read aloud its contents
  - ✓ Explain the contents to a class of students.
- **Proplem:** But what happens if you **can't find** the specified book? You can't proceed with the rest of the action without it so you **need to report back** to the person who assigned the task to you. This unexpected event (missing book) prevents you from completing your task. By reporting it back, you want the originator of this request to take corrective or alternate steps.

Using Throws and Throw Statements (cont.)

- **Creating a method that throws a checked exception**

```
public class DemoThrowsException {
    public void readFile(String file) throws          //#1
                    FileNotFoundException {
     boolean found = findFile(file);
     if (!found)
     throw new FileNotFoundException("Missing file"); //#2
     else {
                  //code to read file
              }
         }
    boolean findFile(String file) {
    // code to return true if file can be located
    }
}
```

Using Throws and Throw Statements (cont.)

- **In which:**
  - **#1:** The throws statement indicates that this method can throw `FileNotFoundException`
  - **#2:** If file can't be found, code creates and throws an object of FileNotFoundException by using the **`throw`** statement
  - A method can include names of multiple, **comma** separated names in its throws statement
- **Using a method that throws a checked exception**

  When you use a method that throws a checked exception
  - Enclose the code within a try block and catch the thrown exception.
  - Declare the exception to be rethrown in the method's signature.
  - Implement both the above together.

Using Throws and Throw Statements (cont.)

- Enclose the code within a try block and catch the thrown exception:

```
void useReadFile(String name) {
    try {
      readFile(name);
    } catch (FileNotFoundException e) {
      // code
    }
}
```

- Declare the exception to be rethrown in the method's signature:

```
void useReadFile(String name) throws
                FileNotFoundException{
          readFile(name);
}
```

Using Throws and Throw Statements (cont.)

- **Implement both the above together:**

```
void useReadFile(String name) throws
                FileNotFoundException {
    try {
      readFile(name);
    } catch (FileNotFoundException e) {
      // code
    }
}
```

Checked vs Unchecked exceptions

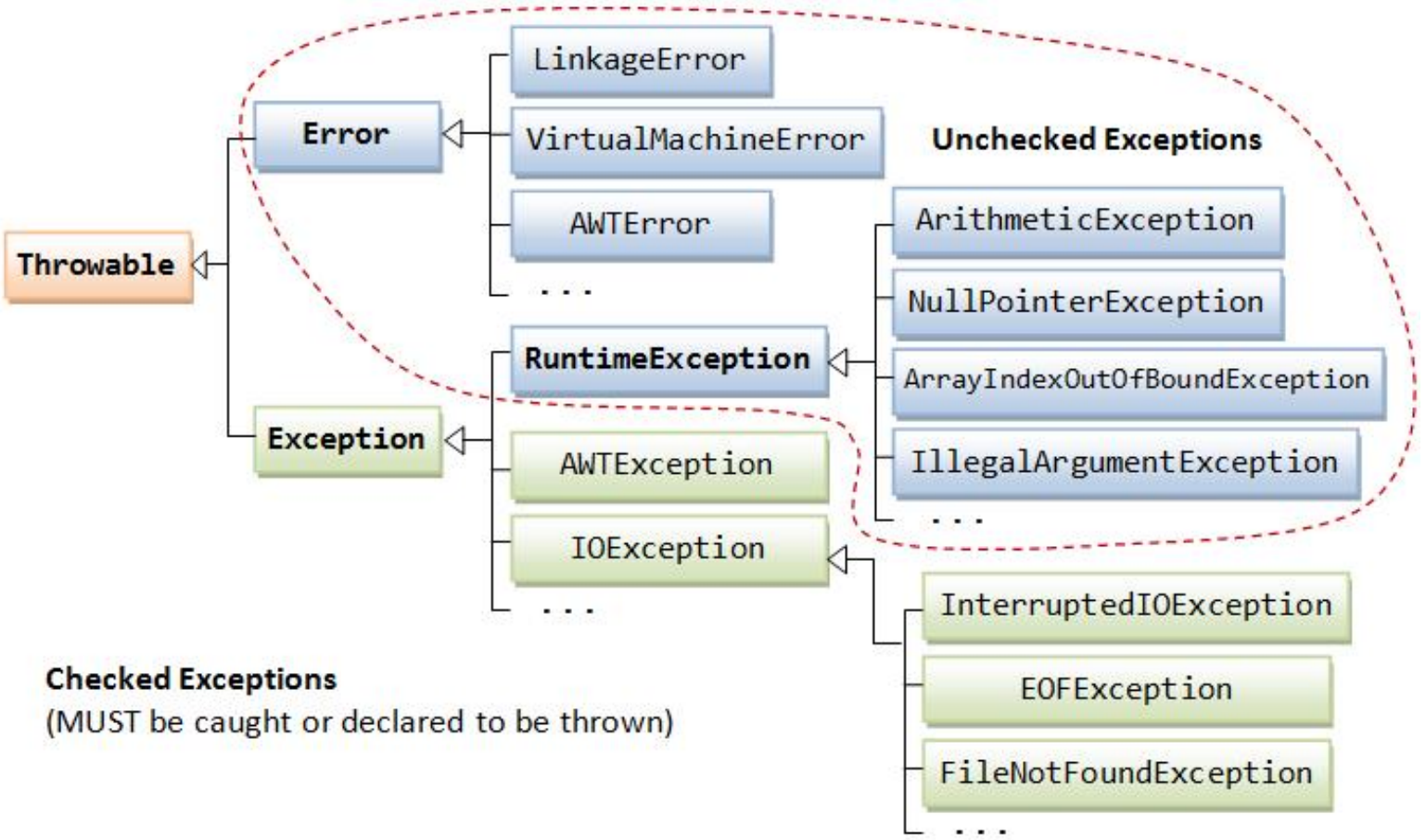
- **Checked exceptions**
  - All exceptions other than Runtime Exceptions are known as Checked exceptions as the **compiler checks them during compilation** to see whether the programmer has handled them or not.
  - If these exceptions are not handled/declared in the program**, it will give compilation error**.
  - **Examples of Checked Exceptions:**
    - `ClassNotFoundException`
    - `IllegalAccessException`
    - `NoSuchFieldException`
    - `EOFException, etc.`
- **See ex1**

Checked vs Unchecked exceptions (cont.)

- **Unchecked Exceptions**
  - Runtime Exceptions are also known as Unchecked Exceptions as the **compiler do not check whether** the programmer has handled them or not but it's the duty of the programmer to handle these exceptions and provide a safe exit.
  - If these exceptions are not handled/declared in the program**, it will not give compilation error**.
  - **Examples of UnChecked Exceptions:**
    - `ArithmeticException`
    - `ArrayIndexOutOfBoundsException`
    - `NullPointerException`
    - `NegativeArraySizeException, etc.`
- **See ex1**

Exception classes

- The figure below shows the hierarchy of the Exception classes.
  - The base class for all Exception objects is: `java.lang.Throwable`, `java.lang.Exception` and `java.lang.Error`.



Summary

- Java IO Basic
- Exception Handling

Thank you